

Rapport Projet : **Network Science and Graph Learning**



Fait par : Mounir AMANOUS

05 mai 2025

Question 1: Reading

Documents lus :

1. Jacobs et al. (2015)
2. Traud et al. (2011)
3. Traud et al. (2012)

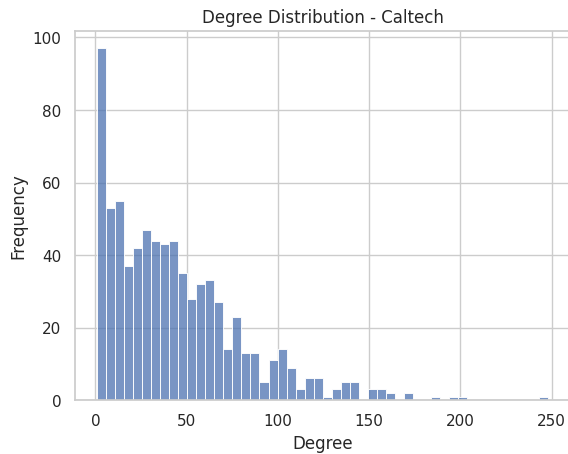
Question 2: Social Network Analysis with the Facebook100 Dataset

(a) Degree Distribution

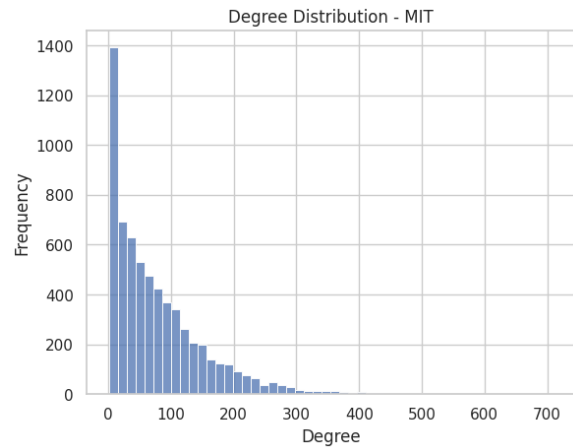
Code :

```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 # Dictionnaire des fichiers      charger
6 target_universities = {
7     'Caltech': 'Caltech36.gml',
8     'MIT': 'MIT8.gml',
9     'Johns Hopkins': 'Johns Hopkins55.gml'
10 }
11
12 # Calcul et affichage de la distribution des degrés
13 degrees = [d for n, d in G.degree()]
14 plt.figure()
15 sns.histplot(degrees, bins=50, kde=False)
16 plt.title(f'Degree Distribution - {name}')
17 plt.xlabel('Degree')
18 plt.ylabel('Frequency')
19 plt.grid(True)
20 plt.show()
```

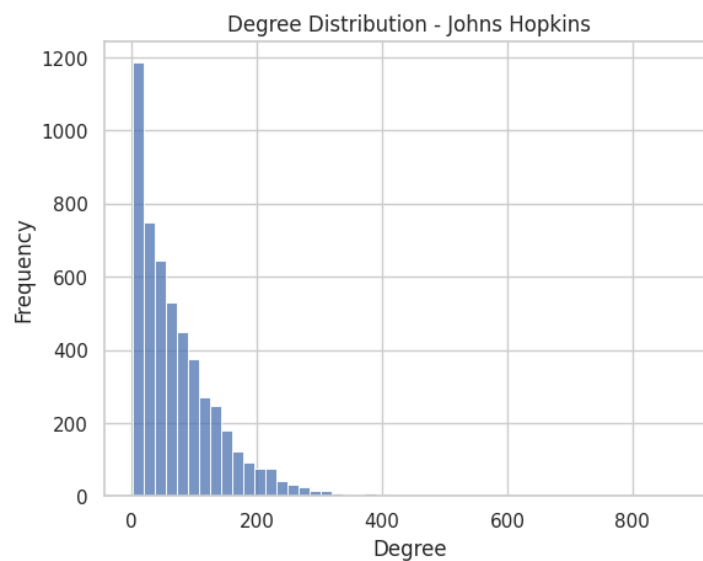
Commentaire sur la figure 1 : On remarque que la fréquence d'un degré donné est élevée quand le degré est faible et inversement. Cela indique que la majorité des nœuds ont peu de connexions, tandis qu'une minorité a un degré très élevé (des "hubs").



(a) Distribution des degrés – Caltech



(b) Distribution des degrés – MIT



(c) Distribution des degrés – Johns Hopkins

Figure 1: Distributions des degrés pour trois universités

(b) Clustering Coefficient and Density

Code :

```

1 for name, G in graphs.items():
2     global_clustering = nx.transitivity(G)
3     avg_local_clustering = nx.average_clustering(G)
4     edge_density = nx.density(G)
5
6     print(f"{name}")
7     print(f"    - Global clustering coefficient: {global_clustering:.4f}")
8     print(f"    - Mean local clustering coefficient: {avg_local_clustering:.4f}")
9     print(f"    - Edge density: {edge_density:.6f}")
10    print()

```

Analyse du clustering et de la densité

Le tableau 1 présente les coefficients de clustering et la densité des arêtes pour les trois réseaux étudiés : Caltech, MIT et Johns Hopkins.

Table 1: Clustering et densité des réseaux pour Caltech, MIT et Johns Hopkins

Réseau	Clustering global	Clustering local moyen	Densité
Caltech	0.2913	0.4091	0.0574
MIT	0.1803	0.2724	0.0123
Johns Hopkins	0.1932	0.2690	0.0140

Interprétation :

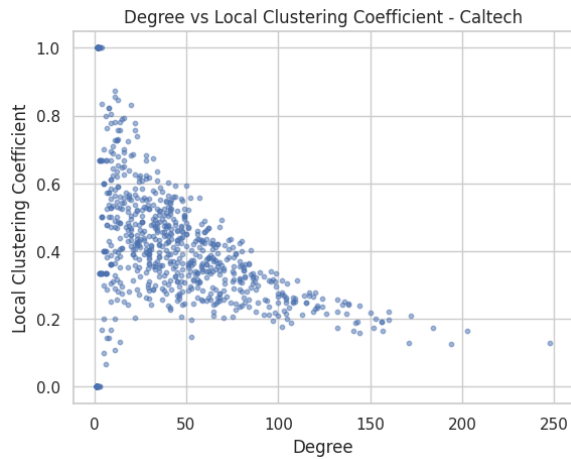
- **Sparsité** : Tous les réseaux ont une densité faible (< 0.06), ce qui signifie qu'ils sont *sparse* : peu d'arêtes comparé au nombre de connexions possibles.
- **Clustering** : Le réseau de Caltech présente un clustering global et local significativement plus élevé que les deux autres, suggérant une structure communautaire plus marquée (cercles sociaux plus denses et plus cohésifs).
- **Topologie** : Les faibles densités combinées à des clustering relativement élevés sont caractéristiques des réseaux sociaux réels : peu de connexions globales mais des sous-groupes fortement connectés (effet de monde petit).

(c) Degree vs. Local Clustering

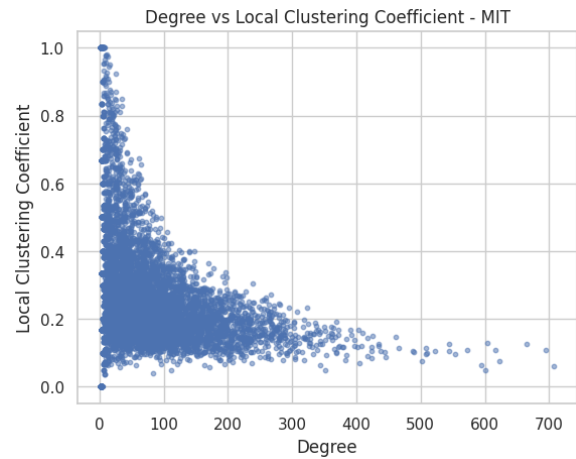
Code :

```
1
2 for name, G in graphs.items():
3     degrees = dict(G.degree())
4     clustering = nx.clustering(G)
5     degree_values = []
6     clustering_values = []
7
8     for node in G.nodes():
9         degree_values.append(degrees[node])
10        clustering_values.append(clustering[node])
11
12    plt.figure()
13    plt.scatter(degree_values, clustering_values, alpha=0.5, s
14               =10)
15    plt.title(f'Degree vs Local Clustering Coefficient - {name}')
16    plt.xlabel('Degree')
17    plt.ylabel('Local Clustering Coefficient')
18    plt.grid(True)
19    plt.show()
```

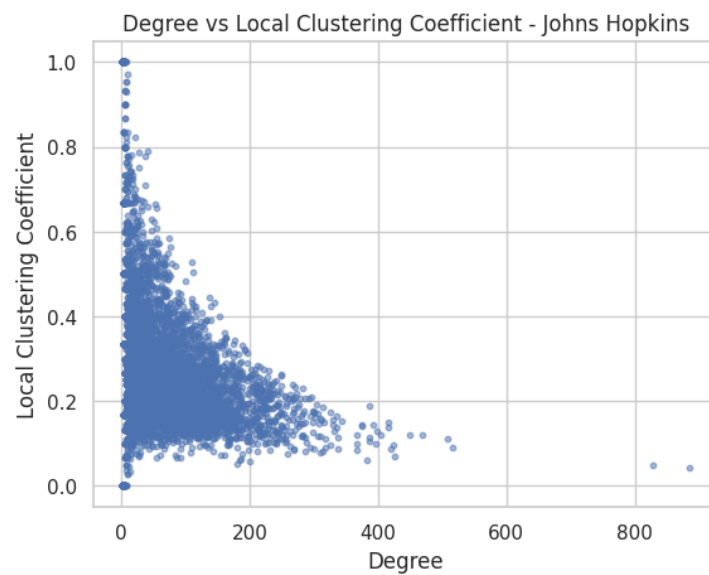
Commentaires sur la figure 2 : On observe souvent une tendance inverse : plus le degré est élevé, plus le clustering local est faible. Cela peut indiquer la présence de hubs connectés à des nœuds peu connectés entre eux.



(a) Degree vs local Clustering – Caltech



(b) Degree vs local Clustering – MIT



(c) Degree vs local Clustering – Johns Hopkins

Figure 2: Distributions des degrés pour trois universités

Question 3: Assortativity Analysis

Code :

```

1 # Dictionnaire des fichiers      charger
2 universities = {
3     'MIT': 'MIT8.gml',
4     'Johns Hopkins': 'Johns Hopkins55.gml',
5     'Stanford': 'Stanford3.gml',
6     ...
7     'Texas': 'Texas80.gml',
8 }
9 # vertex attributes
10 attributes = ['student_fac', 'major_index', 'dorm', 'gender', '
    degree']

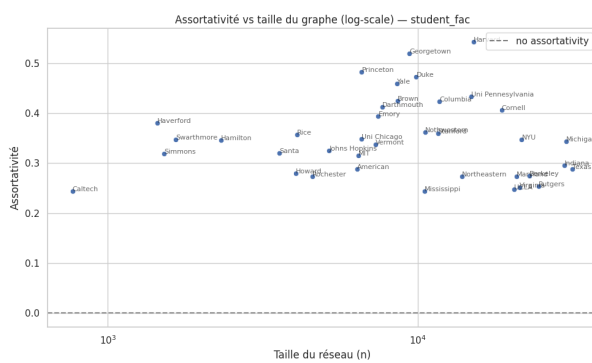
```

```

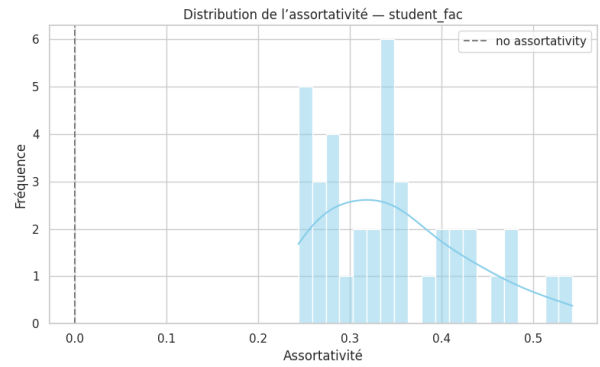
11
12 # results
13 results = []
14 for name, G in q3_graphs:
15     for attr in attributes:
16         if attr == 'degree':
17             assort = nx.degree_assortativity_coefficient(G)
18         else:
19             assort = nx.attribute_assortativity_coefficient(G,
20                 attr)
21     results.append({
22         'university': name,
23         'attribute': attr,
24         'assortativity': assort,
25         'n_nodes': G.number_of_nodes(),
26     })

```

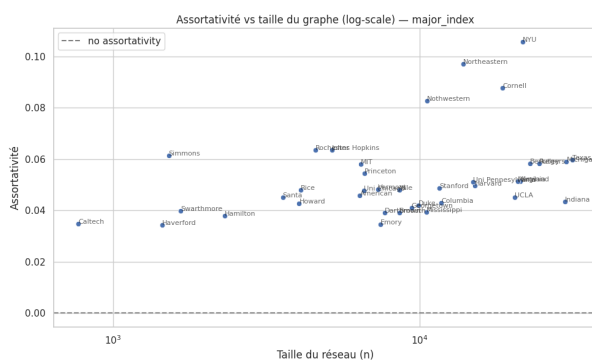
Visualisation des coefficients d'assortativité pour divers attributs :



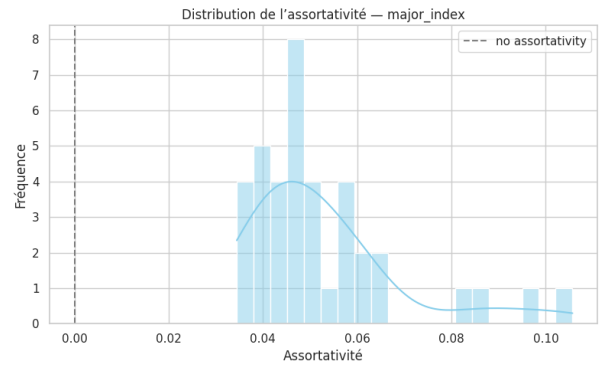
(a) Scatter – Statut



(b) Distribution – Statut

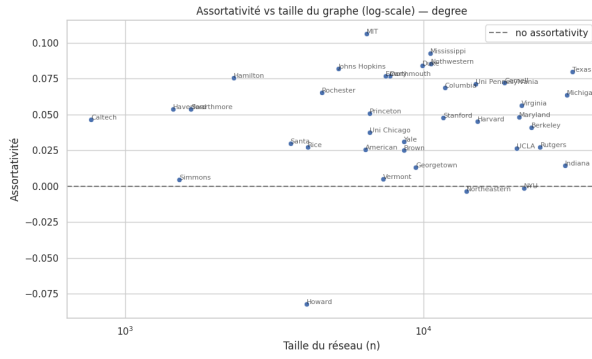


(c) Scatter – Filière

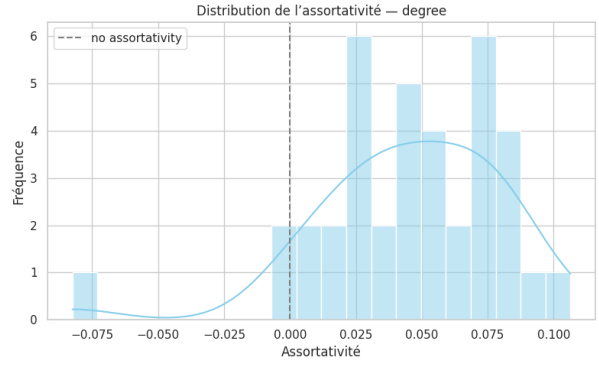


(d) Distribution – Filière

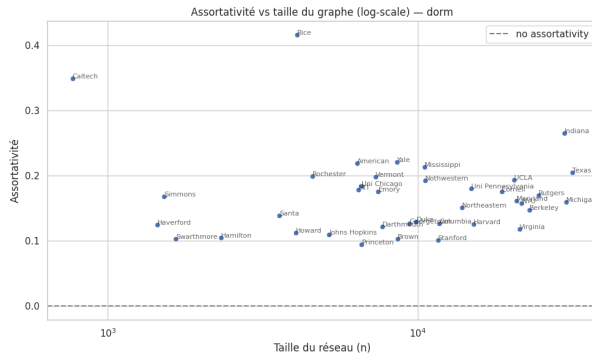
Figure 3: Assortativité – Statut et Filière



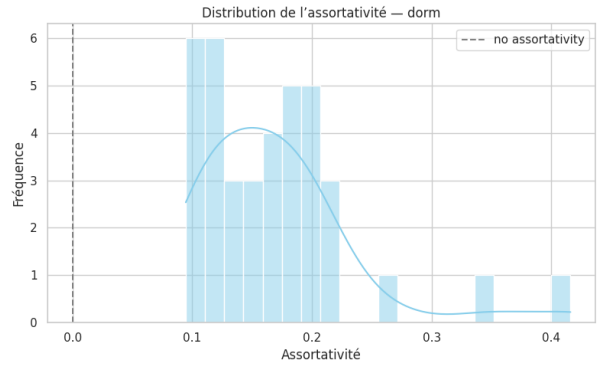
(a) Scatter – Degree



(b) Distribution – Degree

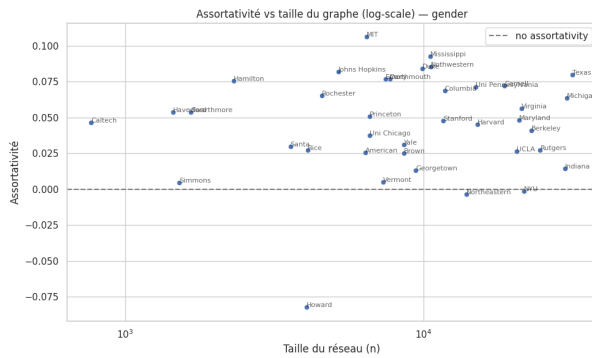


(c) Scatter – Dortoir

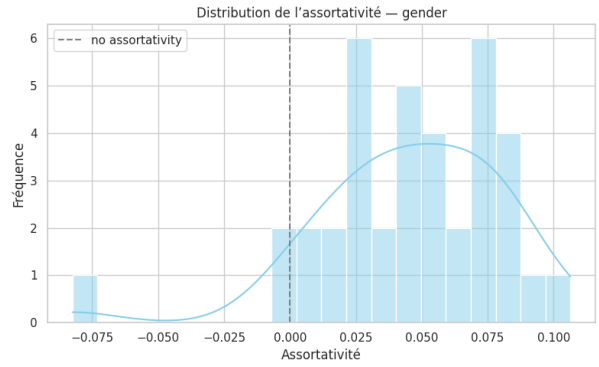


(d) Distribution – Dortoir

Figure 4: Assortativité – Diplôme et Dortoir



(a) Scatter – Genre



(b) Distribution – Genre

Figure 5: Assortativité – Genre

Analyse de l'assortativité selon différents attributs :

De manière générale, on observe que plus la taille des réseaux augmente, plus l'assortativité tend à croître, bien que cette tendance ne soit pas toujours vérifiée pour tous les attributs.

Les attributs *student_fac*, *major* et *dorm* présentent une assortativité positive, ce qui indique que les utilisateurs sont plus susceptibles de se lier à d'autres qui partagent ces mêmes caractéristiques. Cela traduit une forte homophilie : les amitiés sur Facebook se forment souvent entre étudiants d'un même dortoir, d'une même spécialité ou du même statut. L'attribut *student_fac* montre une assortativité particulièrement élevée, dépassant de 0.2 celle des autres attributs.

En revanche, les attributs *degree* (nombre de connexions) et *gender* (sexe) montrent une assortativité proche de zéro ou négative dans quelques universités, suggérant un manque de corrélation ou un réseau dissortif.

Question 4: Link Prediction

(a) Lecture

Document : Liben-Nowell et Kleinberg (2003)

(b) Implémentation des métriques

Métriques implémentées : Common Neighbors, Jaccard, Adamic/Adar

```
1 import random
2 import math
3 from abc import ABC, abstractmethod
4
5 # Classe abstraite pour la pr diction de liens
6 class LinkPredictor(ABC):
7     def __init__(self, graph):
8         self.graph = graph
9
10    @abstractmethod
11    def predict(self, node_pairs):
12        pass
13
14 # Impl ementations personnalis es des m triques
15 class CommonNeighborsPredictor(LinkPredictor):
16     def predict(self, node_pairs):
17         scores = []
18         for u, v in node_pairs:
19             common = len(set(self.graph.neighbors(u)) & set(self.
20                 graph.neighbors(v)))
21             scores.append((u, v, common))
22         return scores
23
24 class JaccardPredictor(LinkPredictor):
25     def predict(self, node_pairs):
26         scores = []
27         for u, v in node_pairs:
28             neighbors_u = set(self.graph.neighbors(u))
29             neighbors_v = set(self.graph.neighbors(v))
30             intersection = len(neighbors_u & neighbors_v)
31             union = len(neighbors_u | neighbors_v)
32             score = intersection / union if union else 0
33             scores.append((u, v, score))
34         return scores
35
36 class AdamicAdarPredictor(LinkPredictor):
37     def predict(self, node_pairs):
38         scores = []
39         degree_cache = {}
40         for u, v in node_pairs:
```

```

40         common_neighbors = set(self.graph.neighbors(u)) & set
           (self.graph.neighbors(v))
41         score = 0
42         for z in common_neighbors:
43             if z not in degree_cache:
44                 degree_cache[z] = len(list(self.graph.
           neighbors(z)))
45             degree = degree_cache[z]
46             if degree > 1:
47                 score += 1 / math.log(degree)
48         scores.append((u, v, score))
49     return scores

```

(c) Évaluation des prédictions

```

1  # Fonction d' valuation optimis e
2  def evaluate_predictor(G, predictor_class, f=0.1, k_values=[50,
           100, 200, 400], max_samples=100000):
3      # Cr ation de la copie du graphe
4      G_train = G.copy()
5
6      # S paration des ar tes
7      edges = list(G.edges())
8      random.shuffle(edges)
9      n_remove = int(len(edges) * f)
10     removed_edges = set(edges[:n_remove])
11     G_train.remove_edges_from(removed_edges)
12
13     # chantillonnage des non-ar tes
14     non_edges = list(nx.non_edges(G_train))
15     random.shuffle(non_edges)
16     samples = non_edges[:max_samples] + list(removed_edges)
17
18     # Calcul des scores
19     predictor = predictor_class(G_train)
20     scores = predictor.predict(samples)
21
22     # Tri des scores
23     sorted_scores = sorted(scores, key=lambda x: x[2], reverse=
           True)
24
25     # Calcul des m triques
26     results = {}
27     removed_set = {(u, v) for u, v in removed_edges} | {(v, u)
           for u, v in removed_edges}
28
29     for k in k_values:
30         top_k = {(u, v) for u, v, _ in sorted_scores[:k]}
31         true_positives = len(top_k & removed_set)
32

```

```

33         precision = true_positives / k if k > 0 else 0
34         recall = true_positives / len(removed_edges) if
           removed_edges else 0
35
36         results[k] = {'precision': precision, 'recall': recall}
37
38     return results
39
40 # Exemple d'utilisation sur 2 graphes
41 selected_graphs = [('Caltech', q3_graphs[0][1]), ('MIT',
           q3_graphs[1][1])]
42 metrics = {
43     'CommonNeighbors': CommonNeighborsPredictor,
44     'Jaccard': JaccardPredictor,
45     'AdamicAdar': AdamicAdarPredictor
46 }
47
48 results = {}
49
50 for name, G in selected_graphs:
51     print(f" valuation sur {name} ({G.number_of_nodes()} n uds ,
           {G.number_of_edges()} ar tes)")
52     graph_results = {}
53
54     for metric_name, predictor_class in metrics.items():
55         print(f" Calcul {metric_name}...")
56         metric_results = evaluate_predictor(G, predictor_class)
57         graph_results[metric_name] = metric_results
58
59     results[name] = graph_results
60
61 # Visualisation des r sultats
62 def plot_metrics(results, metric_type='precision'):
63     plt.figure(figsize=(12, 6))
64
65     for graph_name, graph_data in results.items():
66         for metric_name, metric_data in graph_data.items():
67             x = list(metric_data.keys())
68             y = [v[metric_type] for v in metric_data.values()]
69             plt.plot(x, y, marker='o', label=f"{graph_name} - {
           metric_name}")
70
71     plt.title(f'Performance des pr dicteurs ({metric_type}@k)')
72     plt.xlabel('k')
73     plt.ylabel(metric_type)
74     plt.xscale('log')
75     plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
76     plt.grid(True)
77     plt.show()

```

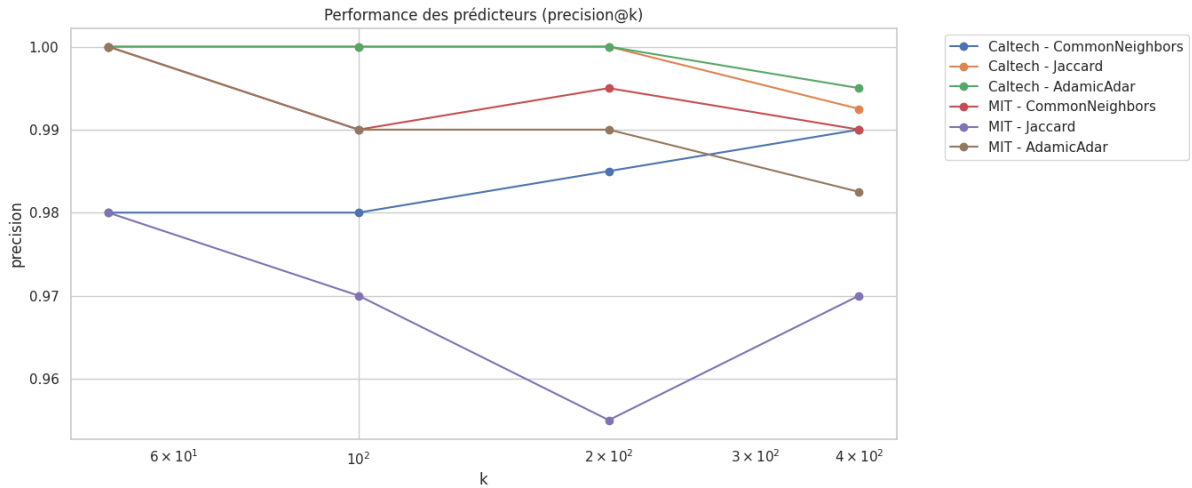


Figure 6: Précision des prédicteurs

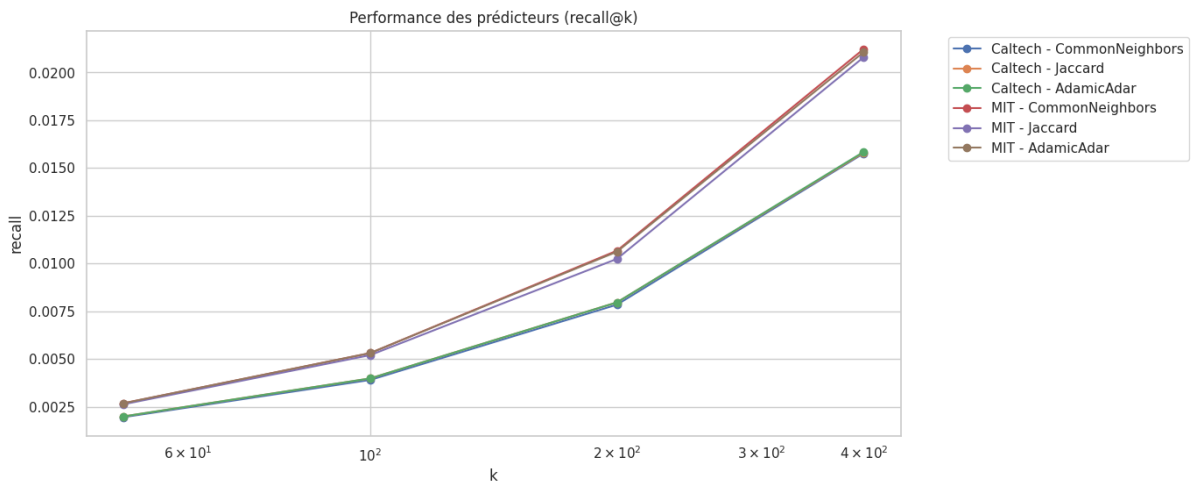


Figure 7: Rappel des prédicteurs

Résultats expérimentaux pour différentes valeurs de k :

Les graphiques de performance des algorithmes de prédiction de liens (précision@k et rappel@k) permettent de comparer trois méthodes — Common Neighbors, Jaccard et Adamic/Adar — appliquées aux réseaux sociaux de Caltech et du MIT.

D'un côté, le graphique de précision montre que la performance tend à diminuer à mesure que k augmente, ce qui est attendu car un plus grand nombre de prédictions inclut des résultats moins fiables. Parmi les méthodes testées, Adamic/Adar se distingue par une précision élevée et stable, notamment sur le réseau de Caltech, indiquant une grande fiabilité de ses prédictions. À l'inverse, Jaccard s'avère plus sensible et moins performant, en particulier sur le réseau de MIT.

Du côté du rappel, on observe une tendance inverse : plus k augmente, plus le rappel s'améliore, car davantage de liens réels sont capturés. Common Neighbors semble obtenir un bon compromis entre précision et rappel, ce qui peut en faire un choix pertinent si l'on souhaite maximiser le nombre de liens correctement identifiés.

Ces résultats soulignent l'importance de choisir l'algorithme en fonction de l'objectif poursuivi : si l'on cherche à minimiser les faux positifs, Adamic/Adar est à privilégier; si l'on vise à identifier un maximum de liens existants, Common Neighbors peut être plus adapté. Par ailleurs, les différences de performances entre Caltech et MIT révèlent des caractéristiques structurelles propres à chaque réseau, comme une densité ou une homogénéité locale différente, qui influencent directement l'efficacité des prédicteurs.

Question 5: Label Propagation

(a–b) Algorithme utilisé :

```

1 def label_propagation(G, labels, max_iter=500):
2     A = nx.to_numpy_array(G)
3     n = A.shape[0]
4
5     # Associer chaque n ud un index (0 n-1)
6     node_list = list(G.nodes())
7     node_to_idx = {node: idx for idx, node in enumerate(node_list
8         )}
9
10    # Matrice de transition
11    D = np.diag(A.sum(axis=1))
12    P = np.zeros((n, n))
13    for i in range(n):
14        for j in range(n):
15            if A[i, j] > 0:
16                P[i, j] = A[i, j] / D[i, i]
17
18    # Encodage des labels
19    unique_labels = list(set(labels.values()))
20    label_to_index = {label: idx for idx, label in enumerate(
21        unique_labels)}
22    Y = np.zeros((n, len(unique_labels)))
23
24    for node_id, label in labels.items():
25        if node_id in node_to_idx:
26            idx = node_to_idx[node_id]
27            Y[idx, label_to_index[label]] = 1
28
29    # Propagation avec limite d'it ration
30    for _ in range(max_iter):
31        Y_new = np.dot(P, Y)
32        if np.array_equal(np.argmax(Y, axis=1), np.argmax(Y_new,
33            axis=1)):

```

```

31         break
32     Y = Y_new
33
34     # Conversion inverse
35     index_to_label = {v: k for k, v in label_to_index.items()}
36     predicted_indices = np.argmax(Y, axis=1)
37     predicted_labels = [index_to_label[i] for i in
38                         predicted_indices]
39
40     # Retourner un dict avec les vrais IDs de n ud
41     return {node: predicted_labels[node_to_idx[node]] for node in
42            G.nodes()}
43
44 # On utilise le graphe de Duke pour l'exemple afin de
45 correspondre celui de l' nonc
46 nx_graph = nx.read_gml(data_folder / universities['Duke'], label=
47 'id')
48 nx_graph = nx.Graph(nx_graph) # Convertir en graphe non orient
49
50 # On cr trois graphe en supprimant respectivement 10%, 20% et
51 30% des noeuds
52 def remove_nodes(G, percentage):
53     num_nodes_to_remove = int(len(G.nodes()) * percentage)
54     nodes_to_remove = np.random.choice(G.nodes(),
55     num_nodes_to_remove, replace=False)
56     G.remove_nodes_from(nodes_to_remove)
57     return G
58 nx_graph_10 = remove_nodes(nx_graph.copy(), 0.1)
59 nx_graph_20 = remove_nodes(nx_graph.copy(), 0.2)
60 nx_graph_30 = remove_nodes(nx_graph.copy(), 0.3)
61
62 LPA_10_dorm = label_propagation(nx_graph_10, nx.
63 get_node_attributes(nx_graph_10, 'dorm'))
64 LPA_20_dorm = label_propagation(nx_graph_20, nx.
65 get_node_attributes(nx_graph_20, 'dorm'))
66 LPA_30_dorm = label_propagation(nx_graph_30, nx.
67 get_node_attributes(nx_graph_30, 'dorm'))
68 LPA_10_major = label_propagation(nx_graph_10, nx.
69 get_node_attributes(nx_graph_10, 'major_index'))
70 LPA_20_major = label_propagation(nx_graph_20, nx.
71 get_node_attributes(nx_graph_20, 'major_index'))
72 LPA_30_major = label_propagation(nx_graph_30, nx.
73 get_node_attributes(nx_graph_30, 'major_index'))
74 LPA_10_gender = label_propagation(nx_graph_10, nx.
75 get_node_attributes(nx_graph_10, 'gender'))
76 LPA_20_gender = label_propagation(nx_graph_20, nx.
77 get_node_attributes(nx_graph_20, 'gender'))
78 LPA_30_gender = label_propagation(nx_graph_30, nx.
79 get_node_attributes(nx_graph_30, 'gender'))
80
81 def accuracy_score(y_true, y_pred):

```

```

67     correct = sum(1 for true, pred in zip(y_true, y_pred) if true
68                  == pred)
    return correct / len(y_true)

```

(c–d) Résultats sur Duke :

Tableau des scores (Accuracy / MAE) :

Table 2: Précision de la propagation de labels pour différents attributs selon le pourcentage de données manquantes

Pourcentage	Dortoir	Filière	Genre
10%	0.294745	0.200876	0.442174
20%	0.288530	0.197954	0.444543
30%	0.193200	0.207449	0.440162

(e) Analyse :

Les résultats montrent que la précision de la propagation de labels varie selon l'attribut :

- **Genre** : meilleure précision (~44%), stable même avec 30% de données manquantes. Cela suggère que les connexions sociales sont souvent homogènes en genre.
- **Dortoir** : performance moyenne, en baisse avec plus de données manquantes. L'information de dortoir est partiellement corrélée aux structures du graphe.
- **Filière** : précision faible (~20%), relativement constante. L'attribut major semble peu reflété dans les liens du réseau.

En conclusion, la propagation de labels fonctionne mieux pour les attributs fortement liés à la structure sociale (comme le genre), et moins bien pour les attributs plus dispersés ou indépendants des relations sociales.

Question 6: Community Detection

(a) Question de recherche

Hypothèse : Les étudiants partageant le même dortoir ("dorm") sont plus susceptibles d'appartenir à la même communauté détectée structurellement dans le graphe d'amitiés.

(b) Code et résultats

```
1 from networkx.algorithms.community import
  label_propagation_communities
2 from collections import defaultdict
3
4 def detect_communities(G):
5     # Utilise l'algorithme de propagation de labels natif de
      NetworkX
6     comms = list(label_propagation_communities(G))
7     # Retourne un dict {node: community_id}
8     node2comm = {}
9     for i, comm in enumerate(comms):
10         for node in comm:
11             node2comm[node] = i
12     return node2comm
13
14 def compute_homogeneity(G, node2comm, attr):
15     # Mesure la proportion de paires dans une meme communauté qui
      partagent le meme attribut
16     attr_dict = nx.get_node_attributes(G, attr)
17     comm2nodes = defaultdict(list)
18     for node, cid in node2comm.items():
19         comm2nodes[cid].append(node)
20     same_attr, total = 0, 0
21     for nodes in comm2nodes.values():
22         for i in range(len(nodes)):
23             for j in range(i+1, len(nodes)):
24                 n1, n2 = nodes[i], nodes[j]
25                 if attr_dict.get(n1) is not None and attr_dict.
                   get(n2) is not None:
26                     total += 1
27                     if attr_dict[n1] == attr_dict[n2]:
28                         same_attr += 1
29     return same_attr / total if total > 0 else 0
30
31 # Calcul sur plusieurs universit s
32 indices = [30, 31, 32, 33, 34] # 5 graphes consecutifs      partir
      de l'indice 30
33 results = []
34
35 for idx in indices:
36     name, G = q3_graphs[idx]
37     node2comm = detect_communities(G)
38     score = compute_homogeneity(G, node2comm, 'dorm')
39     results.append((name, score))
40
41 mean_score = sum(score for _, score in results) / len(results)
```


(c) Conclusion

Table 3: Taux d'homogénéité

Université	Taux d'homogénéité
Michigan%	0.251143
Rice%	0.158633
Rochester%	0.186866
Rutgers%	0.356592
Santa%	0.162946
Moyenne%	0.223236

Une moyenne de 0.223236 montre que les communautés détectées ne correspondent pas aux dortoirs: l'appartenance à un dortoir n'est pas le facteur dominant de structuration des groupes d'amis dans ce réseau Facebook universitaire.